tapmaan.

Climate & Heatwave Monitoring and Early Warnings

Python mini project report | Academic year 2026-27

One Python project. Two interfaces.
An interactive browser dashboard and a Tkinter desktop application, using common Python classes, an SQLite database and a small linear-regression learning model.

Programme / year	Information Technology / SY
Semester / course	III / OOP-PY (216U04L301)
Project use case	Climate and heatwave monitoring with early warnings
Core technologies	Python, Tkinter, SQLite, HTML, CSS and JavaScript
Prepared on	23 October 2026

Team details - complete before submission

Member	Name	Roll number
1	_____	_____
2	_____	_____
3	_____	_____
4	_____	_____
5, if applicable	_____	_____

Lab batch: _____ Faculty: _____

The supplied notice asks for 4-5 students from the same lab batch and the same AI use case. Confirm the group with the lab faculty. This report does not assign or certify contributions.

Problem, objectives & scope

Problem statement

Weather readings are difficult to interpret when they are scattered across sources. A student-friendly monitoring tool can collect temperature and humidity, store readings, show the near-term temperature outlook and highlight high-temperature conditions before they occur.

Project objectives

- Present current weather and a 24-hour temperature outlook for six Indian cities.
- Save observations and warning-review states in a relational SQLite database.
- Demonstrate a small learning model that estimates the next three-hour temperature trend.
- Provide a Tkinter GUI and an interactive website with a shared Python backend.
- Keep network requests off the Tkinter event loop using a worker thread and queue.

Mapping to the supplied assessment instructions

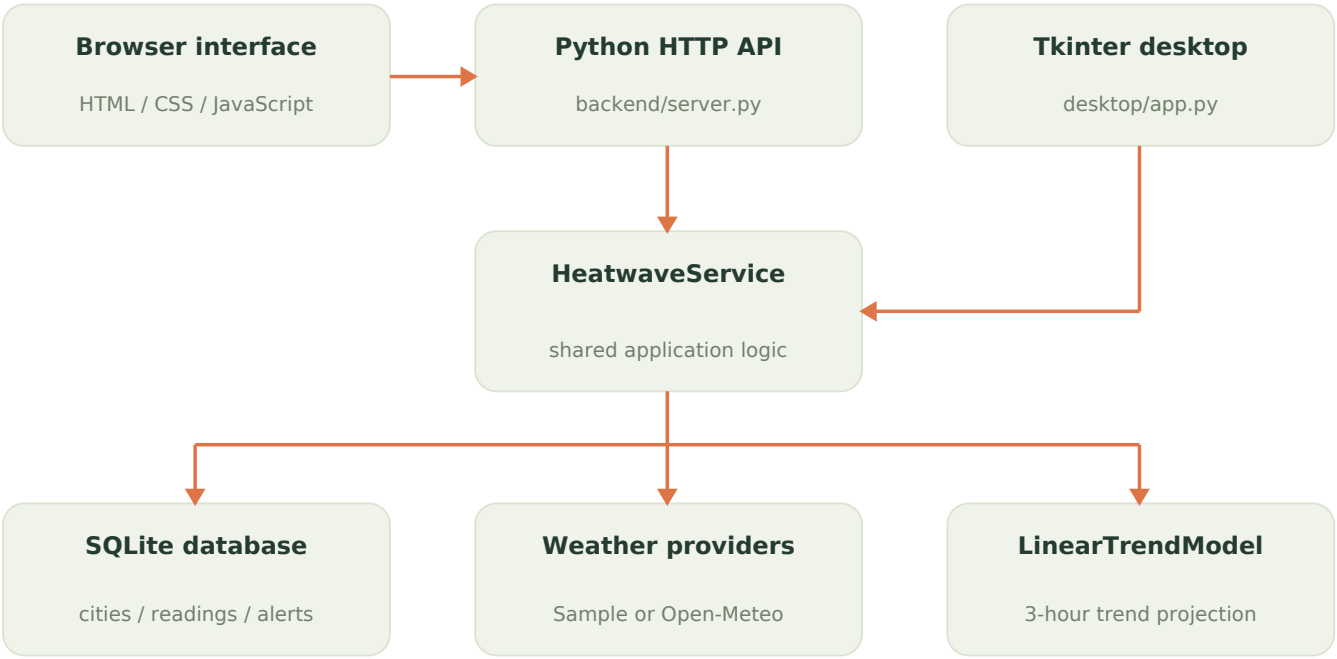
Requirement	Implementation
Python OOP	WeatherReading, WeatherProvider, HeatwaveService, Database and LinearTrendModel classes
Tkinter GUI	desktop/app.py: forms, city selector, chart, tables and callbacks
SQLite connectivity	database/schema.sql + backend/database.py + database/tapmaan.db
Multithreading	Tkinter worker thread + queue; threaded HTTP request handling
Report and viva	This report, CODE_GUIDE.md and DEMO_GUIDE.md
Mini-project marks	Problem/design: 3; implementation/demo: 5; documentation/viva: 2

Scope: This is a local educational prototype for short-term weather monitoring. It does not implement long-term climate trend analysis, official heatwave declaration, SMS/email delivery or unattended monitoring.

Source: the supplied OOP-PY laboratory assessment PDF and the accompanying Python mini-project instruction image.

System design & data flow

The web interface uses a local JSON API. The desktop interface calls the same service directly. The service isolates weather fetching, prediction and storage so that the two interfaces stay consistent.



Example: saving a manual observation

1. The user enters temperature, humidity and time in the website form.
2. JavaScript sends JSON to POST /api/readings.
3. Python validates the values and creates a WeatherReading object.
4. Database.save_many() inserts the row using SQL placeholders and commits.
5. The API returns the saved ID and risk; the website reloads the table.
6. Tkinter can read that same row from SQLite after Reload saved data.

Preview distinction: The hosted page is an interactive sample preview with browser storage. The downloadable local project runs the real Python HTTP API and SQLite database. Tkinter is a separate desktop window, not a web technology.

Database design & SQL

SQLite stores the project in database/tapmaan.db. The schema is created automatically at startup. Three tables keep city information, readings and reviewable warnings separate.

Table	Key columns and purpose
cities	id (PRIMARY KEY), name, region, latitude, longitude. One row per supported city.
readings	id (PRIMARY KEY), city_id (FOREIGN KEY), recorded_at, temperature, humidity, source, kind. Source is demo/live/manual; kind is observation/forecast.
alerts	id (PRIMARY KEY), reading_id (UNIQUE FOREIGN KEY), level, acknowledged. One optional warning per reading.

Relationships and integrity

- One city has many readings. Each reading belongs to one city.
- One reading has zero or one warning. High/Severe values create warning rows.
- A unique key on city + time + source + kind prevents duplicate refresh rows.
- CHECK constraints restrict temperature, humidity, source and kind values.
- Foreign keys are enabled on every connection. Deleting a manual reading also removes its associated warning through ON DELETE CASCADE.

Example query: join observations with their city

```
SELECT c.name, r.recorded_at, r.temperature, r.source
FROM readings r
JOIN cities c ON r.city_id = c.id
WHERE r.kind = 'observation'
ORDER BY r.recorded_at DESC;
```

Parameterized SQL in Python

```
sql = "SELECT * FROM readings WHERE city_id=? AND source=?"
rows = db.execute(sql, ("dombivli", "manual")).fetchall()
```

Placeholders keep user values separate from SQL syntax. Each operation opens its own connection, commits on success, rolls back on failure and closes the connection. WAL mode and a connection timeout support the two local interfaces accessing the same file.

More examples: database/example_queries.sql. The application restricts deletion to manual observations; it does not let users remove provider data through the interface.

Python OOP & multithreading

Concept	Concrete use in this project
Class / object	WeatherReading holds one weather record and validates it.
Abstraction	WeatherProvider declares the abstract fetch(city) method.
Inheritance	DemoProvider and OpenMeteoProvider inherit WeatherProvider; TapmaanApp inherits tk.Tk.
Polymorphism	The selected provider implements fetch(city); service code calls the same method name.
Encapsulation	Database owns SQLite operations; interfaces use its methods.
Composition	HeatwaveService contains its database, providers and learning model.
Exception handling	Invalid input raises ValueError; network failures display a useful error; failed database transactions roll back.

Tkinter: a responsive event loop

The desktop app starts a worker only for weather fetching. The worker does not access Tkinter widgets. It puts a success/error result into queue.Queue. The main thread polls this queue using after(100, ...) and then updates labels and tables.

```
def worker():
    self.service.refresh(city, mode)
    self.results.put(("success", "Weather refreshed"))

threading.Thread(target=worker, daemon=True).start()

# In the main GUI thread:
status, message = self.results.get_nowait()
self.after(100, self.check_results)
```

The excerpt above shows the pattern; the delivered implementation includes try/except handling and busy-state controls. It never shares a SQLite connection across threads.

Website request threads

ThreadingHTTPServer handles HTTP requests in separate threads. This allows the website to request stored data while other requests are being processed. The server binds to 127.0.0.1 for a local classroom demonstration.

Early warnings & the learning model

Temperature	Project category	Database warning
Below 35°C	Normal	No
35°C to below 40°C	Watch	No
40°C to below 45°C	High	Yes
45°C and above	Severe	Yes

These rules use temperature only. Humidity is stored and shown, but it does not affect the classifier. The categories are educational thresholds, not official IMD heatwave criteria. Acknowledging a warning marks it reviewed; it does not reduce the underlying temperature or risk.

Linear regression: what is learned

The model selects at least three observations from the last six hours of the selected source. It converts timestamps to elapsed hours x, uses temperatures as y and fits a straight line by least squares. Forecast rows and manual readings are not mixed into the provider model.

```
slope     = sum((x - mean_x) * (y - mean_y))
            / sum((x - mean_x) ** 2)
intercept = mean_y - slope * mean_x
estimate  = intercept + slope * (last_x + 3)
```

Worked sample: Dombivli fictional scenario

Output	Value / interpretation
Current sample observation	39.1°C at 12:00 IST on 18 May 2026; Watch
Provider outlook peak	41.1°C at 15:00 IST; High
Learning-model projection	44.0°C for 15:00 IST, from 7 recent observations
Why the two differ	The small model extrapolates morning warming as a straight line; the provider scenario has a daily cycle.

No accuracy claim: This model demonstrates fitting and prediction. It has no held-out validation or calibrated uncertainty estimate, and can overshoot when the daily trend changes. The separate 24-hour provider forecast must not be presented as the model's output.

Open-Meteo supplies live current and hourly weather through HTTPS. Its values are model-based weather estimates, not measurements from a sensor connected to this project. Sample mode uses a fixed fictional scenario.

Installation & demonstration

Run the two interfaces

Extract the complete ZIP. Open the project folder in VS Code and use Python 3.10 or newer. No third-party pip packages are needed. The standard Windows Python installation should include Tcl/Tk.

```
python run_web.py
# Open http://127.0.0.1:8000

# In a second terminal:
python run_desktop.py
```

On Windows, py -3 can replace python. The included start_website.bat and start_desktop.bat are shortcuts. Test Tkinter with python -m tkinter. If the web port is busy, use python run_web.py --port 8001.

Suggested five-minute demo

Time	Action and concept to show
0:00-0:40	Introduce the purpose and explain the two interfaces.
0:40-1:30	Use Sample data; change city, compare history/forecast, review a warning.
1:30-2:30	Save 42.5°C / 58% manually; show the saved row, CSV export and the same record in Tkinter.
2:30-3:30	Show Tkinter controls; fetch live weather using the worker thread if internet is available.
3:30-4:30	Open code: provider inheritance, service methods and parameterized SQL.
4:30-5:00	Explain limitations and distinguish the trend model from the provider forecast.

Suggested work division

Person 1: web frontend. Person 2: Python providers and model. Person 3: database and validation. Person 4: Tkinter, threading and integration. Optional Person 5: documentation and testing. Assign roles according to work actually completed; these are suggestions, not an authorship claim.

Classroom reliability: Demonstrate Sample data first. A live API failure preserves the database. The local website and desktop need a running Python process; opening index.html directly does not connect the backend.

Testing, results & limitations

Run the delivered test suite with the following command:

```
python -m unittest discover -s tests -v
```

Check	Observed result
Warning boundaries + bad input	Exact 35/40/45°C transitions and invalid/non-finite values passed.
Database operations	Save/reopen, acknowledgement persistence, unique refresh rows and cascade deletion passed.
Thread safety	Twelve concurrent writes through separate connections passed.
Model and source separation	Known straight-line prediction and insufficient-input behavior passed; manual data did not alter the provider model.
API and failure handling	Real local HTTP GET/POST, malformed JSON, cross-origin rejection and failed live fetch preservation passed.
External weather request	A live Open-Meteo request returned 50 parsed readings during development.
Browser interaction	Dashboard, save, reload persistence, CSV download, warning review, city switching and live-mode explanation checked.
Tkinter window	Source compiled and shared backend tested. Visual launch not possible here: no graphical display. Verify on the student laptop.

Backend result: 13 automated tests passed. Provider-response parsing is also tested with a fixed fixture, so tests do not require internet access.

Practical limitations

- The learning model is a short-term linear extrapolation, not a validated meteorological forecasting system.
- Warning thresholds are temperature-only classroom rules; official alerts require location-specific criteria and appropriate authority data.
- The hosted sample preview cannot demonstrate actual Python or SQLite execution; use the local project for assessment.
- The desktop display must be tested on the student laptop. Web layout was reviewed at desktop size; responsive CSS is included.

Code map, conclusion & references

File / starting line	Purpose
frontend/app.js:16	Frontend -> Python API
backend/server.py:46	JSON request handling
backend/service.py:54	Validation and save flow
backend/database.py:13	Python -> SQLite connection
backend/database.py:36	INSERT/UPDATE + warning records
backend/providers.py:35	Python -> weather API
backend/predictor.py:7	Regression fit and prediction
desktop/app.py:99	Worker thread
desktop/app.py:119	Queue + main-thread UI update

Exact line numbers refer to this delivered version and change if files are edited. The complete 17-location map, API route table and OOP explanation are in docs/CODE_GUIDE.md. English and Hinglish presentation lines are in docs/DEMO_GUIDE.md.

Conclusion

Tapmaan demonstrates how Python OOP, a Tkinter GUI, a browser interface, SQLite storage and multithreading can work together in one understandable project. It connects a climate-related use case with testable program behavior and transparent model limitations.

References

Python Tkinter documentation
https://docs.python.org/3/library/tkinter.html

Python SQLite documentation
https://docs.python.org/3/library/sqlite3.html

Python threading documentation
https://docs.python.org/3/library/threading.html

Python HTTP server documentation
https://docs.python.org/3/library/http.server.html

Open-Meteo weather API documentation
https://open-meteo.com/en/docs

Assignment sources supplied by the user: 216U04L301_OOP-PY_labCA-2026-27.pdf and WhatsApp Image 2026-10-23 at 2.46.35 PM.jpeg. References checked on 23 October 2026.